Apple of my Earth

A Work-in-Progress Report on Approach, Machine Learning, and Authorship

Samuel Flückiger - CAS AI in Creative Practices - University of Bern / ZhdK

This report accompanies a work-in-progress presentation of «Apple of my Earth», a generative short film that follows a single potato plant across three eras and landscapes. The central thread running through the piece is the *protection gap*: the recurring failure of the systems we build to shelter the (living) things that depend on them. The work is being developed for exhibition on a cathode-ray television playing from a VHS tape, and the fragment shown at this stage is the opening act - an Andean valley beneath a volcano, cycling repeatedly through day, dusk, night and dawn. Presenting it on a dying playback medium is deliberate: the image of a fragile landscape held inside an obsolete, degrading technology is a statement about the limits of preservation. It’s also a reflection of my childhood, when I dreamt of becoming a filmmaker after watching lots of Hollywood films on VHS.

Approach

I don't generate the film as finished clips on demand. I build it as a controlled production pipeline with reproducible, versioned assets. Rather than prompting a video model for finished shots and accepting whatever it returns, I separate each moment into layers: a locked landscape plate, separately generated lighting states of that same plate, and - later in the pipeline - a plant, which is always produced as an independent element and composited over the environment rather than baked into it. This separation is the backbone of the whole project. It lets me hold the composition of the valley absolutely fixed while changing only the light, which is what the exhibition demands: the same landscape must pass through many convincing cycles of day and night without the mountains, the volcano or the terraces ever shifting or warping.

To achieve this I built an orchestration layer around three chosen pillars, mirroring the workflow presented in the CAS Imagery module: a reasoning-and-scripting agent (Claude) acting as both script consultant and pipeline architect; a set of Python scripts that execute every generation step; and a node-based editor (ComfyUI) as the execution engine, with the heavy computation running on rented cloud GPUs (RunComfy) rather than my own hardware. Every plate is generated from an explicit specification held in a structured project file, embedding the seed and full metadata into a JSON file for each saved image so that any result can be reproduced or recovered after the fact. The discipline of never overwriting a file, always

incrementing a version, and recording the exact parameters of every generation has been essential; it turned a process that is inherently probabilistic into something I feel I have control over.

This represents a real shift in the project's ambition. I originally intended to chase the newest, highest-performing models available, but shifted toward a different goal: understanding how machine learning actually ties into image generation deeply enough to build my own modular, reproducible pipeline. The trade-off is explicit - by fixing a stable stack (FLUX.2, Wan FLF2V, RIFE, Depth Anything V2) and investing in the architecture around it, I have almost certainly missed newer models released during the project. But what I gained is a pipeline I understand end to end, can reproduce, and can extend across all three acts. For a project this size, that's worth more to me than looking for the latest model.

At the centre of this architecture sits a single orchestration file I call the story beat JSON file (AomE_beats.json). It is, in effect, a machine-readable interpretation of my self-written screenplay: every plate, mask, transition, and sound prompt is encoded there as structured data, and all three Python programs read from it. Producing that translation from screenplay to structured orchestration was an important input from my mentor Mykhailo Vladymyrov, and crucial for my authorship - the creative document remains mine, written by hand in Final Draft, and the pipeline is downstream of it rather than a replacement for it. The programs themselves run as simple commands in the macOS Terminal, each documented in its own manual; there is no browser or third-party login involved, only API keys and deployed workflows, which keeps the whole pipeline self-contained and reproducible.

The role of machine learning

Several distinct machine-learning models do the actual generative work, each chosen for a specific task. A large text-to-image diffusion transformer (FLUX.2) produces the base landscape plates from written descriptions. The same architecture in its image-editing configuration (FLUX.2 edit), driven by a hand-painted inpaint mask, produces the derived states - relighting the valley for dusk, night and dawn, and carving agricultural terraces into the slopes - while a mask protects the regions that must not change. For motion, a first-last-frame video model (Wan FLF2V) interpolates between two locked plates to create the transitions between times of day, and a frame-interpolation model (RIFE) smooths the result into continuous movement. Alongside these, I built a small preprocessing pipeline that generates monocular depth maps of the plates (Depth Anything V2), used as an additional structural conditioning signal in the editing step (discussed under Next Steps). For sound, a family of text-to-audio models (ElevenLabs) generates everything the film

expresses acoustically - atmospheres, discrete sound effects, and an original musical score - each produced from a written description.

Each of these models is a different kind of trained system, and part of the project has been learning to speak to each in its own terms. The diffusion models that produce and edit the images were trained on vast collections of pictures paired with descriptions; they generate by starting from noise and progressively resolving it toward something that matches the text and the surrounding pixels. The masked editing model additionally respects black-and-white mattes, treating white as the region it may change and black as pixels it must return untouched — which is what will allow me to alter one slope of a valley while leaving the rest mathematically identical. The video model was trained on moving footage and understands time, camera behaviour and physical motion; given a first and last frame it invents the plausible movement between them. The interpolation model, on the other hand, is a narrower tool that simply manufactures intermediate frames to raise the smoothness and frame rate of an existing clip.

These systems have no memory between generations, so every instruction has to be absolute - a fixed value, never «more» or «less» than last time. Once I stopped fighting their tendencies and started working with them, the tools became controllable. A large part of my real work has been diagnostic: reading why a model did what it did, forming a hypothesis, and adjusting the constraints so that its behaviour aligns with my intent.

Learnings

The central lesson is that control comes from architecture, not wording. The instinct to search for a better prompt is usually misplaced; with generative tools, authorship lives in how the problem is decomposed and constrained - what is locked, what is layered, and what is exposed to the model versus withheld from it. A film made this way is held together by the structure built around the models, not by the sentences fed into them.

A second lesson is that every model has a hard ceiling, and the craft lies in recognising it early rather than refining against it indefinitely. Much of the practical work has been diagnosing the point where a tool stops being able to do what I need, then switching instruments or adding a second stage, instead of forcing a single model past its limit.

Generation is probabilistic, so reproducibility has to be built in. Capturing the exact conditions of every result, never overwriting, and designing for recovery turns an unpredictable process into one I

can navigate. That discipline is what makes real authorship over the output possible.

Lastly, each model responds to something specific, and progress comes from working with that grain rather than against it. Even failure was informative: a result that reveals why a model behaves as it does is a finding, and not necessarily a dead end.

Next steps

The clearest technical frontier ahead is geometric control of the landscape edits. The film needs the valley to change state - pristine slopes becoming agricultural terraces, and later a scorched, post-eruption ground - while the underlying shape of the hillside, its contour and the way it recedes into the distance, stays fixed. My current masked-edit tool controls *where* a change happens but not how faithfully the model respects the existing three-dimensional form within that region, so the mountain tends to drift as terraces are carved. I have been working to solve this by adding structural conditioning to the edit: giving the model a depth map of the original slope (generated by my Depth Anything V2 preprocessing pipeline) as an explicit constraint on its geometry, alongside the inpaint mask that already governs where the edit may occur. An early attempt to supply that depth map through FLUX.2's native reference-latent mechanism failed - a useful negative result, since it showed that reference-latent conditioning operates with appearance, not spatial structure. I have since moved to a dedicated depth ControlNet (FLUX.2-dev-Fun-ControlNet-Union), which is designed for this kind of structural guidance; it is wired and installed, currently blocked on a compatibility bug between the community-built ComfyUI node and a recent multi-GPU change in the core software, and is the immediate next thing to resolve. The three inputs to this editing step - the pristine plate (image 1), the hand-painted slope mask (image 2), and the generated depth map (image 3) - are shown in the appendix.

Once that structural control is reliable, the same tool produces the two remaining land states - terraced and scorched - without the composition drifting, and the path to completing the first act opens up: generating the full sequence of the valley's transformation, introducing the plant as the protagonist composited over these stable environments, and extending the same layered, versioned pipeline from the opening act to the two eras that follow. The architecture is built to scale this way - what is proven on Act 1 is meant to carry directly into Acts 2 and 3.

Ethical considerations

I set out with a clear ethical intention: to build the entire pipeline from open-source models, whose weights, training data, and licensing can be inspected and reasoned about, rather than closed commercial systems. The core of the pipeline holds to this - FLUX.2, Wan FLF2V, RIFE, and Depth Anything V2 are all open-source. The audio, however, is where I broke my own rule. Under deadline pressure, and after the open video-to-audio route failed for my material, I adopted a closed commercial API (ElevenLabs) because it was the only option that delivered professional results in the time I had. I want to name this honestly rather than hide it, because it is a small instance of a much larger pattern: the consensus in a FOCAL ComfyUI course I did to support my CAS work was the same uncomfortable conclusion I reached in practice - that working with open-source models alone is, for now, not enough to stay competitive or to deliver professional output on a real timeline. The VFX industry is pivoting toward ethically problematic closed models precisely because they win on quality and speed, and individual good intentions bend under the deadline pressure I felt. My breach is minor in isolation; as a microcosm of where the field is heading, it is the part of this project I am least comfortable with, and I'd rather mention it openly.

My contribution and authorship

It matters to me to be clear about what is mine in a project made with generative tools. The models generate pixels; the film is authored in the decisions around them. The concept, the three-era structured screenplay, and the protection-gap theme are mine. So is every compositional and creative choice: the locked camera, the decision to show humans only as partial figures etc. And lastly, the exhibition itself as a degrading VHS loop on a cathode-ray screen. The production architecture - the layered pipeline, the masking strategy, the versioning discipline, the recovery mechanisms - is my design, arrived at through sustained iteration and diagnosis. The LLM assistant I work with proposes options and writes code and prompts to my specification, but the creative and architectural authority remains with me. I'm not demonstrating a tool. I'm using one, carefully, in service of a specific cinematic idea.

The fragment shown at the CAS work-in-progress exhibition - the opening valley moving through its cycles of day and night on VHS tape - is an honest snapshot of a work still in progress. The lighting states of Act 1 are locked, the first transitions between them are working, and the remaining pieces (the cultivated and scorched states of the land, the plant as protagonist, and the ambient sound) are underway. It's unfinished, but I hope the method and the idea behind it already come through.

Appendix A — Skills built and models used

Custom skills and tools I built using an LLM/agent (Claude):

generate_plate.py - Generates landscape plates via the cloud API in two modes - text-to-image for fresh base plates, and masked image-editing for derived states (relights, terracing). Embeds the seed and full metadata into every saved image sidecar file, supports batch generation, and recovers gracefully from network interruptions.

generate_transition.py — Produces the motion between two locked plates using a first-last-frame video model, then smooths and extends the result with a frame-interpolation model to reach a target duration - a solution I proposed after hitting the video model's hard native length ceiling. Includes quality presets (preview, master) so cheap iteration and one final full-quality render are cleanly separated, plus recovery of interrupted cloud jobs.

generate_audio.py — Generates all of the film's sound from text prompts through a family of ElevenLabs models, in four modes: atmospheres, discrete sound effects, an original musical score, and voice. Pulls prompts from the orchestration file, versions every result, and peak-normalises each to a consistent level (this replaced an earlier video-to-audio approach that gave weaker results).

Machine-learning models used in the pipeline:

- *Text-to-image diffusion model (Flux.2)* - generates the base landscape plates from written descriptions.
- *Image-editing diffusion model (Flux.2 edit, mask-driven)* - produces derived states — relighting for dusk, night and dawn, and carving terraces — while a hand-painted mask protects the regions that must not change.
- *First-last-frame video model (Wan)* - interpolates believable motion between two locked plates to create the transitions between times of day.
- *Frame-interpolation model (RIFE)* - manufactures intermediate frames to raise smoothness and frame rate.
- *Text-to-audio models (ElevenLabs)* - generate atmospheres, sound effects, an original score, and voice — each from a written description.
- *Depth ControlNet (Flux.2 Fun ControlNet-Union) - in progress* - a structural-conditioning model that constrains an edit to preserve

the slope's existing geometry, for producing the terraced and scorched land states without the mountain drifting.

Appendix B - Deliverables for the CAS exhibition

Engine demo on VHS - for this presentation, an edited demo video showcasing the working pipeline - plate generation, transitions, and sound - exported to VHS and played through a cathode-ray television. The degrading playback medium forms part of the work's meaning (the full Act 1 timelapse loop is the eventual exhibition form; the demo stands in for it at this stage).

Screenplay in paper form - a printed screenplay placed beside the screen, readable by the exhibition visitor, giving the written companion to the moving image.

Printed project documentation & skill manual - beside the screen, a set of printed documents that together show how the work is made: the project reference document, the skills manual, and the beat-JSON file that orchestrates the whole pipeline.

Appendix C - Images

The following images document the three inputs to the geometry-preserving editing step and are referenced in the Next Steps section.

Image 1 — pristine Andean valley plate

Image 2 — hand-painted left-slope inpaint mask

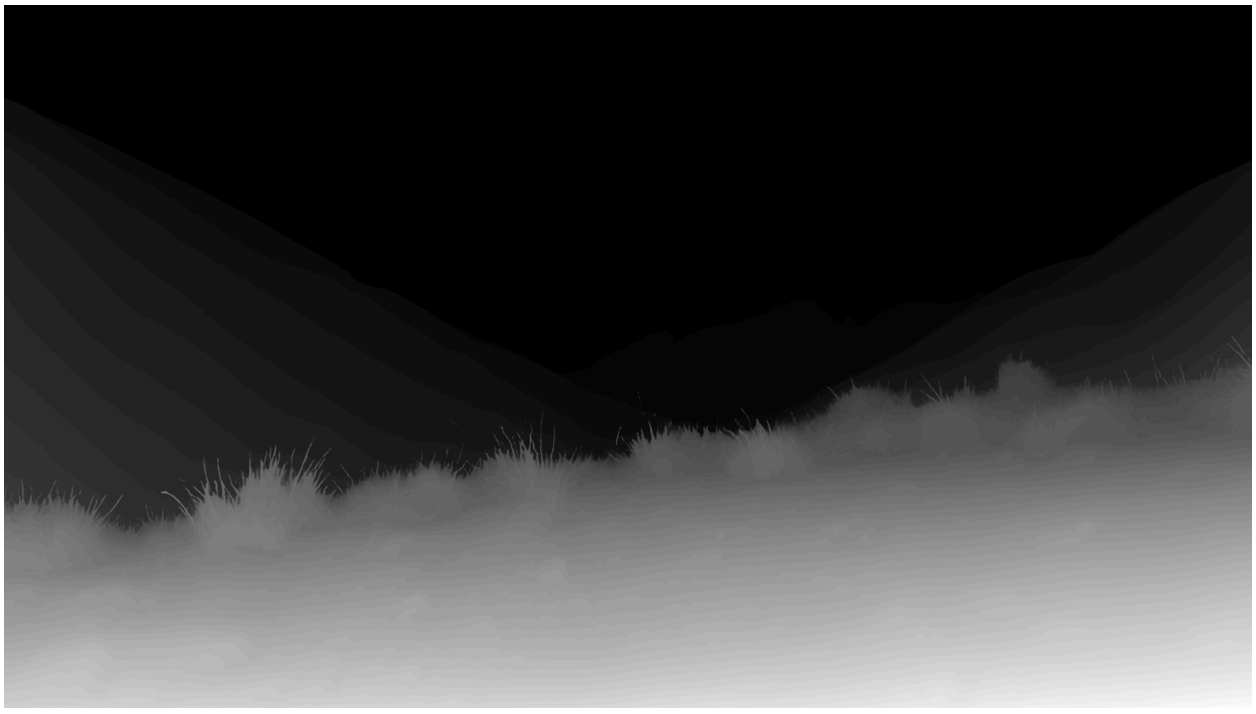

Image 3 — generated monocular depth map (Depth Anything V2)

References

FLUX.2 — Black Forest Labs. FLUX.2 text-to-image and image-editing diffusion models. https://blackforestlabs.ai

Wan (FLF2V) — Wan-AI. Wan first-last-frame video generation model. https://huggingface.co/Wan-AI

RIFE — Huang et al. Real-Time Intermediate Flow Estimation for video frame interpolation. https://github.com/hzwer/ECCV2022-RIFE

Depth Anything V2 — Yang et al. Depth Anything V2 monocular depth estimation. https://depth-anything-v2.github.io

FLUX.2-dev-Fun-ControlNet-Union — Alibaba PAI. https://huggingface.co/alibaba-pai/FLUX.2-dev-Fun-Controlnet-Union

ElevenLabs — Text-to-Sound-Effects, Music, and Text-to-Speech APIs. https://elevenlabs.io

MMAudio — Cheng et al. Video-to-audio synthesis (evaluated and not adopted). https://github.com/hkchengrex/MMAudio

ComfyUI — node-based diffusion execution environment. https://github.com/comfyanonymous/ComfyUI

RunComfy — cloud GPU deployment for ComfyUI workflows. https://www.runcomfy.com